

Problem Statement: AI-Powered Restaurant Recommendation System (Zomato Use Case)

You are tasked with building an **AI-powered restaurant recommendation service** inspired by Zomato. The system should intelligently suggest restaurants based on user preferences by combining structured data with a Large Language Model (LLM).

Objective

Design and implement an application that:

- Takes user preferences (such as location, budget, cuisine, and ratings)
- Uses a real-world dataset of restaurants
- Leverages an LLM to generate personalized, human-like recommendations
- Displays clear and useful results to the user

System Workflow

1. Data Ingestion

- Load and preprocess the Zomato dataset from Hugging Face (<https://huggingface.co/datasets/ManikaSaini/zomato-restaurant-recommendation>)
- Extract relevant fields such as restaurant name, location, cuisine, cost, rating, etc.

2. User Input

- Collect user preferences:
 - Location (e.g., Delhi, Bangalore, Kolkata)
 - Budget (low, medium, high)
 - Cuisine (e.g., Italian, Chinese)
 - Minimum rating
 - Any additional preferences (e.g., family-friendly, quick service)

3. Integration Layer

- Filter and prepare relevant restaurant data based on user input
- Pass structured results into an LLM prompt
- Design a prompt that helps the LLM reason and rank options

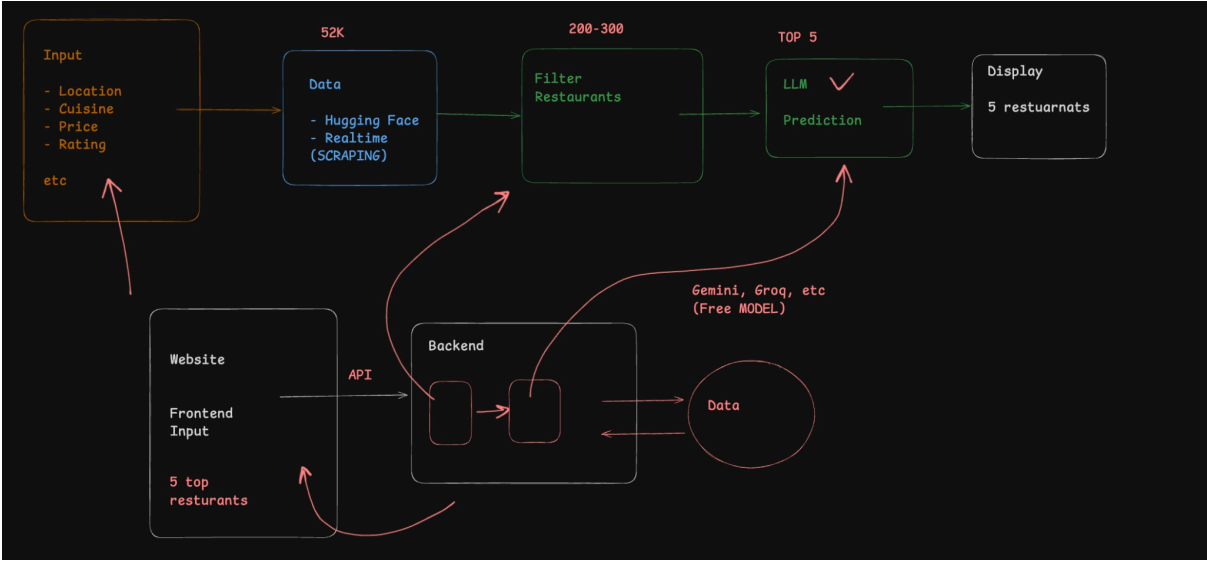
4. Recommendation Engine

- Use the LLM to:
 - Rank restaurants
 - Provide explanations (why each recommendation fits)
 - Optionally summarize choices

5. Output Display

- Present top recommendations in a user-friendly format:
 - Restaurant Name
 - Cuisine
 - Rating
 - Estimated Cost
 - AI-generated explanation

Diagram →



Architecture

Phase-wise architecture: restaurant recommendation system

This document breaks the build into **phases** that map to the workflow in [problemstatement.md](#): data ingestion → user input → integration (filter + prompt prep) → LLM recommendation → output display.

Phase 0 — Scope and foundations

Item	Outcome
Product slice	Basic web UI — source of user input and primary presentation of results for milestone 1 (see phase0-scope.md); CLI remains for dev/diagnostics.
Stack	Language/runtime, dependency manager, where secrets live (e.g. .env for API keys, never committed).
Dataset contract	Confirm Hugging Face dataset fields you will support in v1; document column → internal field mapping.
Non-goals	Explicitly defer (e.g. user accounts, live Zomato API, maps) to avoid scope creep.

Exit criteria: written assumptions (stack, v1 UI, supported preference fields) and a local way to run the app end-to-end once later phases exist.

Implemented artifacts: [phase0-scope.md](#), [dataset-contract.md](#), repo [README.md](#), [.env.example](#), and CLI milestone1 info / milestone1 doctor.

Phase 1 — Data ingestion and canonical model

Layer	Responsibility
Acquisition	Download or stream ManikaSaini/zomato-restaurant-recommendation; cache locally if useful for iteration.
Normalization	Clean types (ratings as numbers, cost as enum or numeric band), handle missing values, dedupe rows if needed.
Canonical schema	Internal Restaurant (or equivalent) with: name, location, cuisines, cost, rating, plus any extra columns you keep for prompts.

Exit criteria: a single module (or package) that loads data and returns a typed in-memory collection or queryable table; unit tests on parsing for a few sample rows.

Implemented: package folder `src/milestone1/ingestion/` (Restaurant, `load_restaurants` / `iter_restaurants`, normalization, Hub revision pin, schema assertion). CLI: `milestone1 ingest-smoke --limit N`. Hub integration tests: `RUN_HF_INTEGRATION=1 pytest -m integration`.

Phase 2 — User preferences and validation

Component	Responsibility
Preference model	Structured fields: location, budget band, cuisine(s), minimum rating; optional free-text for “additional preferences.”

Validation	Reject or coerce invalid input (unknown location, rating out of range); clear error messages for the UI/CLI.
-------------------	--

Exit criteria: preferences deserialize from form/API/CLI args into one object used by the filter layer; validation errors are user-visible.

Implemented: package folder src/milestone1/preferences/ (UserPreferences, preferences_from_mapping, optional allowed_city_names corpus check, allowed_cities_from_restaurants). CLI: milestone1 prefs-parse ... (prints JSON or field errors on stderr).

Phase 3 — Integration layer (retrieval + prompt assembly)

Component	Responsibility
Deterministic filter	Apply hard filters first: location, min rating, budget, cuisine overlap—reduce to top <i>N</i> candidates (cap for LLM context, e.g. 15–50).
Ranking hint (optional)	Pre-sort by rating or composite score so the LLM sees a sensible default order even before reasoning.
Prompt builder	System + user messages (or single structured prompt) including: user preferences as JSON or bullets; candidate table as markdown/JSON; instructions to only recommend from the list; output format (see Phase 4).

Exit criteria: given preferences + loaded dataset, produce a stable (candidates[], prompt_payload) without calling the LLM yet; tests for filter edge cases (no matches, too many matches).

Phase 4 — Recommendation engine (LLM)

Concern	Approach
---------	----------

Model I/O	Thin client: temperature, max tokens, timeout; inject API key from environment.
------------------	---

Grounding	Prompt requires the model to cite restaurant names from the candidate list only; refuse or return empty if nothing fits.
------------------	---

Structured output	Ask for JSON (e.g. rankings[] with restaurant_id, rank, explanation) or strict markdown sections—then parse and validate.
--------------------------	---

Resilience	Retry on transient errors; fallback: return deterministic top- <i>k</i> with template explanations if the LLM fails.
-------------------	--

Exit criteria: end-to-end call returns ranked items with explanations; parser validates structure; failures degrade gracefully.

Phase 5 — Output and experience

Surface	Responsibility
---------	----------------

Rendering	For each recommendation: name, cuisine, rating, estimated cost, AI explanation (per problem statement).
------------------	---

Empty states	“No restaurants match filters” vs “LLM could not justify picks”—different copy.
---------------------	---

Observability (light)	Log latency, token usage if available, and filter counts (no PII in logs unless required).
------------------------------	--

Exit criteria: demo path from user input to readable results in one run; copy and layout match the minimum fields in the problem statement.

Phase 6 — Hardening and handoff (optional but recommended)

- Automated tests for filters, prompt shape, and JSON parsing (fixtures with fake LLM responses).

- README: how to install, set OPENAI_API_KEY (or provider of choice), run, and limitations.
- Cost/latency notes: candidate cap, model choice, caching strategy for repeated queries.

Phase dependency order

Phases **0–2** can overlap slightly (e.g. stub UI while data loads), but **3** should not depend on the LLM until filters are correct; **4** should not own business rules that belong in **3**.

Traceability to the problem statement

Problem statement section	Primary phase
Data ingestion	1
User input	2
Integration layer	3
Recommendation engine	4
Output display	5

This keeps the architecture **phase-wise** for planning and milestones while preserving a **layered** runtime design (data → rules → model → presentation).

STEPS

STEP 1 → Created a doc folder inside which we added a [problemstatement.md](#)

STEP 2 → Generating the architecture

STEP 3 → Generating the edge cases

Prompts

Prompt 1 → Write @docs/problemstatement.md in better way

Prompt 2 → Create a phase wise architecture for this @docs/problemstatement.md

Prompt 3 → Generate the detailed edge cases for this project using @docs/problemstatement.md and @docs/phased-architecture.md

Prompt 4 → Implement phase0 as per the @docs/phased-architecture.md

Prompt 5 → In phase0 we can update that basic web UI will be the source of input

Prompt 6 → Implement phase1 in the separate folder as per the @docs/phased-architecture.md

Prompt 7 → run phase1 so that data is downloaded

Prompt 8 → Implement phase2 in the separate folder as per the @docs/phased-architecture.md

Prompt 9 → Implement phase3 as per the @docs/phased-architecture.md in the separate folder

Prompt 10 → Implement phase4 as per the @docs/phased-architecture.md in the separate folder, LLM used in this phase will be Openai.

STEPS for creating API KEY on OPENAI:

- Create account on OpenAI:
<https://openai.com/>
- Create an API KEY on:
<https://platform.openai.com/api-keys>
- Add API key in ENV file

Create an .env file YOURSELF :

- OPENAI_API_KEY =
sk-proj-B68gyY9rn-_9PI5BXYMpf5f
RaEgb7AXIUJv17-QZ82nhfMx2yzbs
FufZe0YwhVBalgaJAznyUmT3BIbkF
J_YVLADvYXyu_E5MEI2iraiOgTHKE
hXbjFcVZx8E5zrvschCPoCsPod0rLz
urTYUwL7abPP5zIA

Prompt 11 → Test phase4 will a live example like this below - Input is Bellandur, Budget is 2000, rating is 4.0

Get the top5 restaurants from the LLM

